

Geometry-dependent skin effect

This application illustrates how to identify the geometry-dependent skin effect (GDSE) via numerical calculation of the strip generalized Brillouin zone (SGBZ). The example of interest is the gain-loss Haldane model, which is well acknowledged by its hybrid skin-topological boundary states [1]. It is generally recognized that the bulk bands of the gain-loss Haldane model do not exhibit non-Hermitian skin effect, as its amoebic generalized Brillouin zone (amoebic GBZ) is identical to the Brillouin zone (BZ). However, according to its SGBZs with respect to different major axes, it is manifest that the model exhibits GDSE.

The accompany Python script demonstrates how to:

-

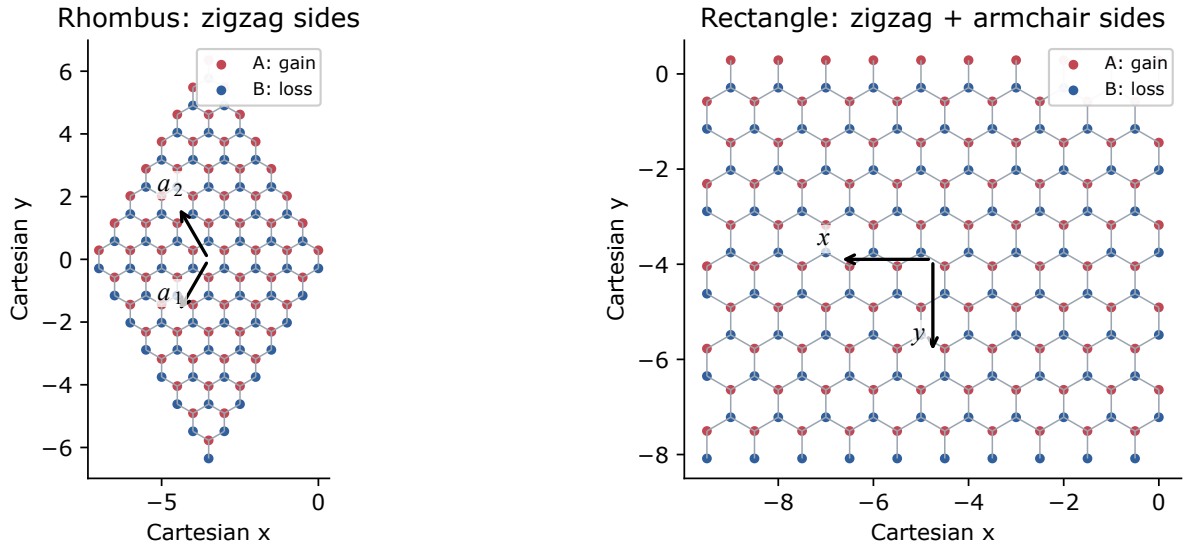


Figure 1: Two open geometries built from the same hopping model. Red and blue sites carry gain and loss, respectively. Arrows denote the lattice directions used by the script; their positive senses need not coincide with positive Cartesian axes. The rhombus has sides parallel to a_1 and a_2 ; the rectangle has sides parallel to x and y . The drawings show small lattices; the OBC data below use 12,800 sites in each geometry.

The accompanying Python script covers BerryPy modeling, directional SGBZ energy scans, saved-data visualization, finite open-boundary diagonalization, and extraction of compact plotting data from older calculations. Its default run is a small demonstration; the figures in this note use existing, larger research datasets.

What “no skin effect” means here The comparison concerns **bulk** states. It does not exclude topological edge states or their corner accumulation. A directional SGBZ is also a sequential thermodynamic limit, rather than an exact prediction for every eigenvalue of a finite sample [2].

BerryPy model and direction conventions

Use a two-site primitive cell with lattice vectors and Cartesian site positions

$$A = (\mathbf{a}_1, \mathbf{a}_2) = \begin{pmatrix} -\frac{1}{2} & -\frac{1}{2} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{3}}{2} \end{pmatrix}, \quad r_A = \left(0, \frac{1}{2\sqrt{3}}\right), \quad r_B = \left(0, -\frac{1}{2\sqrt{3}}\right).$$

The parameters follow the supplied Haldane scripts:

$$t_1 = 1, \quad t_2 = 0.5, \quad \varphi = \frac{\pi}{3}, \quad M = 0.5 \text{ i}, \quad \gamma = 0.$$

Here M and $-M$ are the onsite potentials; γ is the common next-nearest-neighbor hopping phase in the older script, not the gain-loss strength.

BerryPy represents a hopping as `[source, destination, amplitude, cell_shift]`. The corresponding Bloch matrix element contains $\beta_1^{-R_1} \beta_2^{-R_2}$ for cell shift (R_1, R_2) . At $\gamma = 0$, define

$$Q = \beta_1 + \beta_2 + (\beta_1 \beta_2)^{-1}, \quad \tilde{Q} = \beta_1^{-1} + \beta_2^{-1} + \beta_1 \beta_2.$$

The model is

$$h(\beta_1, \beta_2) = \begin{pmatrix} M + t_2(e^{i\varphi} Q + e^{-i\varphi} \tilde{Q}) & t_1(1 + \beta_2 + \beta_1^{-1}) \\ t_1(1 + \beta_2^{-1} + \beta_1) & -M + t_2(e^{-i\varphi} Q + e^{i\varphi} \tilde{Q}) \end{pmatrix}.$$

`build_model()` enters these bonds into `BerryPy.TightBinding.TightBindingModel` and converts the Cartesian site positions with `cart2lattice`. The BZ calculation calls `get_bulk_Hamiltonian_complex(exp(1j * theta))`, with phases in radians. BerryPy's `get_bulk_Hamiltonian(k)` instead expects fractional reciprocal coordinates and internally multiplies them by 2π .

■ Choosing the strip axis

Let $A' = AT$, where the **columns** of the integer matrix T specify new primitive vectors. The first vector is the strip's major axis, parallel to the boundary under study. BerryPy's `get_supercell` constructs the transformed hopping model before its characteristic polynomial is extracted.

Name	Major axis b_1	Minor axis b_2	Sites/cell
<code>a1</code>	$\mathbf{a}_1$	$\mathbf{a}_2$	2
<code>a2</code>	$\mathbf{a}_2$	$\mathbf{a}_1$	2
<code>x</code>	$\mathbf{a}_1 + \mathbf{a}_2$	$\mathbf{a}_1 - \mathbf{a}_2$	4
<code>y</code>	$\mathbf{a}_1 - \mathbf{a}_2$	$\mathbf{a}_1 + \mathbf{a}_2$	4

```
# The two columns give x and y, respectively.
model_x = build_model().get_supercell(
    [(0, 0), (1, 0)], np.array([[1, 1], [1, -1]]))
coeffs, degs = polynomial_data(model_x)
```

The transformed Bloch factors satisfy $\beta'_j = \beta_1^{T_{1j}} \beta_2^{T_{2j}}$. Thus the phase plots for different directions use different coordinates (and the four-site cells fold the bands). The BZ condition $|\beta'_1| = |\beta'_2| = 1$ is nevertheless unambiguous in every basis.

Scanning and displaying the SGBZ

For each direction, the script obtains the Laurent coefficients of

$$f(E, \beta_1, \beta_2) = \det(EI - h(\beta_1, \beta_2))$$

from BerryPy. Each row of `degs` contains the exponents of (E, β_1, β_2) in that order. It then scans a rectangular mesh in the complex energy plane:

```
from pygbz2d.sgbz import collect_GBZ_subsets

coeffs, degs = polynomial_data(direction_model("y"))
result = collect_GBZ_subsets(coeffs, degs, 1.5 + 0j)
if not result.success:
    print(result.error)  # unresolved, not outside the spectrum
elif result.is_gbz:
    print(result.index, result.subsets)
else:
    print("No SGBZ subset at this energy")
```

The solver locates a zero of the average major-axis winding and imposes the transverse equal-modulus root condition. A vanishing winding plateau is excluded by the solver's plateau check. This example uses the package defaults; it does not impose $\mu_1 = 0$, set transverse radii to one, or remove discrepant samples.

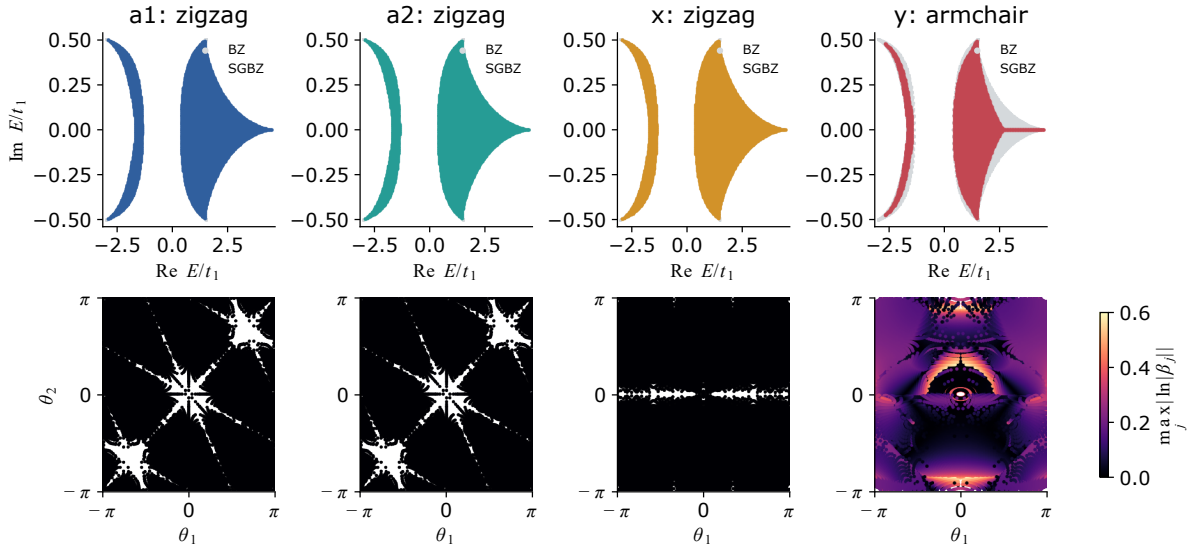


Figure 2: Directional comparison using the existing 201×201 energy scans, with $\text{Re } E \in [-3.1, 4.6]$ and $\text{Im } E \in [-0.51, 0.51]$. Top: sampled SGBZ spectra over the BZ spectrum in gray. Bottom: phases (θ_1, θ_2) of the computed subsets, colored by $\max_j |\ln|\beta_j||$. Every stored line-subset row is included. White regions indicate missing samples in this finite energy scan, not established holes in the BZ or SGBZ. Each panel uses its own lattice basis.

`PointSubset` supplies one pair (β_1, β_2) . For a `LineSubset`, the script uses **all** entries of `theta1_arr` and `beta2_arr`, reconstructing $\beta_1 = \exp(\mu_1 + i\theta_1)$. This matters for the real-energy continuum subsets in the y scan. Failed solves and unfinished checkpoints remain distinguishable from successfully classified spectral exterior points.

Reading the skin-effect diagnostic

Write $\beta_j = \exp(\mu_j + i\theta_j)$. A nonzero $\mu_j = \ln|\beta_j|$ describes exponential growth or decay per cell in direction b_j . The BZ is $\mu_1 = \mu_2 = 0$. Checking only the major-axis radius would miss the effect here: the y -SGBZ has μ_1 close to zero while its transverse μ_2 is generically nonzero. For this strip, b_2 points along x , so the localization is transverse to the armchair boundary.

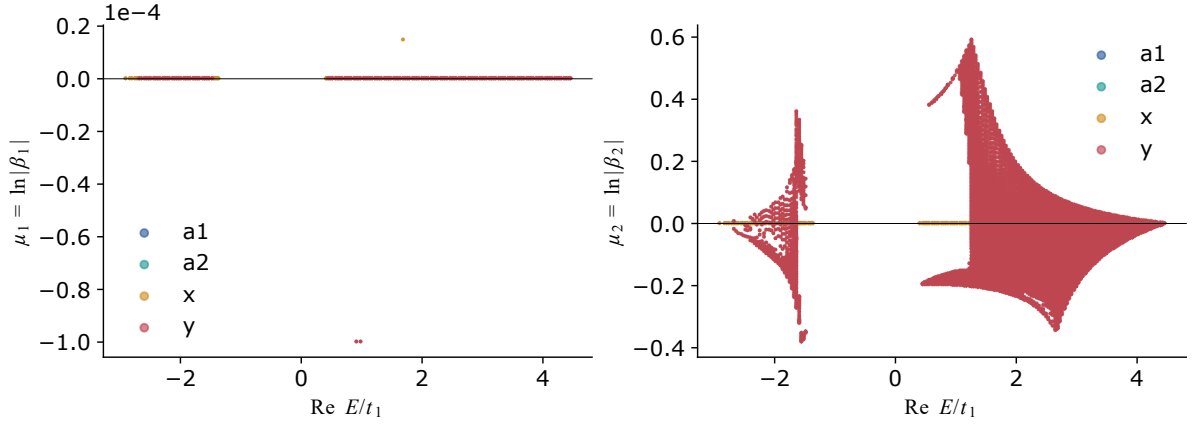


Figure 3: Logarithmic radii of every stored subset sample, plotted against its real energy. The small scale of the major-axis panel reflects numerical deviations near zero. The broad nonzero transverse radii in the armchair case are the main diagnostic; the sample distribution in these plots is not a density of states.

The following statistics are generated by `sweep_report` and stored in `sgbz-summary.json`. Each scan contains 40,401 energies, with no entries marked failed or unfinished.

Direction	In spectrum	Max. $ \mu_1 $	Max. $ \mu_2 $	99th % $ \mu_2 $
a_1	9,991	2.2×10^{-16}	9.15×10^{-10}	2.62×10^{-11}
a_2	9,991	2.2×10^{-16}	9.15×10^{-10}	2.62×10^{-11}
x	9,991	1.46×10^{-5}	1.15×10^{-4}	4.58×10^{-11}
y	6,656	1.00×10^{-4}	0.590	0.505

These samples support a BZ description of the bulk for the three zigzag directions and a non-Bloch description for the armchair direction. The latter is not merely an energy-spectrum comparison: the transverse Bloch factors themselves have nonunit modulus. A finite scan does not establish an exact equality of continuous manifolds, nor does it establish the behavior of every possible polygon.

Numerical deviations retained in the figures

At $E = 1.674 + 0.2397i$, the legacy x scan has six point subsets with $\mu_1 \approx 1.46 \times 10^{-5}$ and maximum $|\mu_2| \approx 1.15 \times 10^{-4}$. A fresh call to the current solver reproduces this deviation. A fixed- $\mu_1 = 0$ winding evaluation is also slightly nonzero (about -1.55×10^{-5}), so this is not simply a plotting or pickle-conversion problem. Its full numerical cause remains unresolved. The figure and maximum statistic retain it; the 99th percentile shows how localized it is in this scan. The small y major-axis deviations are likewise retained.

Constructing finite open geometries

The rhombus repeats the two-site primitive cell over $0 \leq n_1 < N_1$, $0 \leq n_2 < N_2$. Its sides are parallel to $\mathbf{a}_1$ and $\mathbf{a}_2$. The rectangle repeats the four-site $\times$ cell over the same index ranges. Its vertical sides are armchair boundaries, and its horizontal sides are zigzag boundaries. The choice is guided by the directional SGBZ comparison, then checked with finite OBC eigenstates.

`finite_hamiltonian` reads BerryPy's intracell and intercell hopping lists. It retains a bond only when both endpoint cells belong to the finite sample. The right eigenvectors are obtained using the general complex eigensolver `scipy.linalg.eig`; the matrix is non-Hermitian, so `eigh` would be inappropriate.

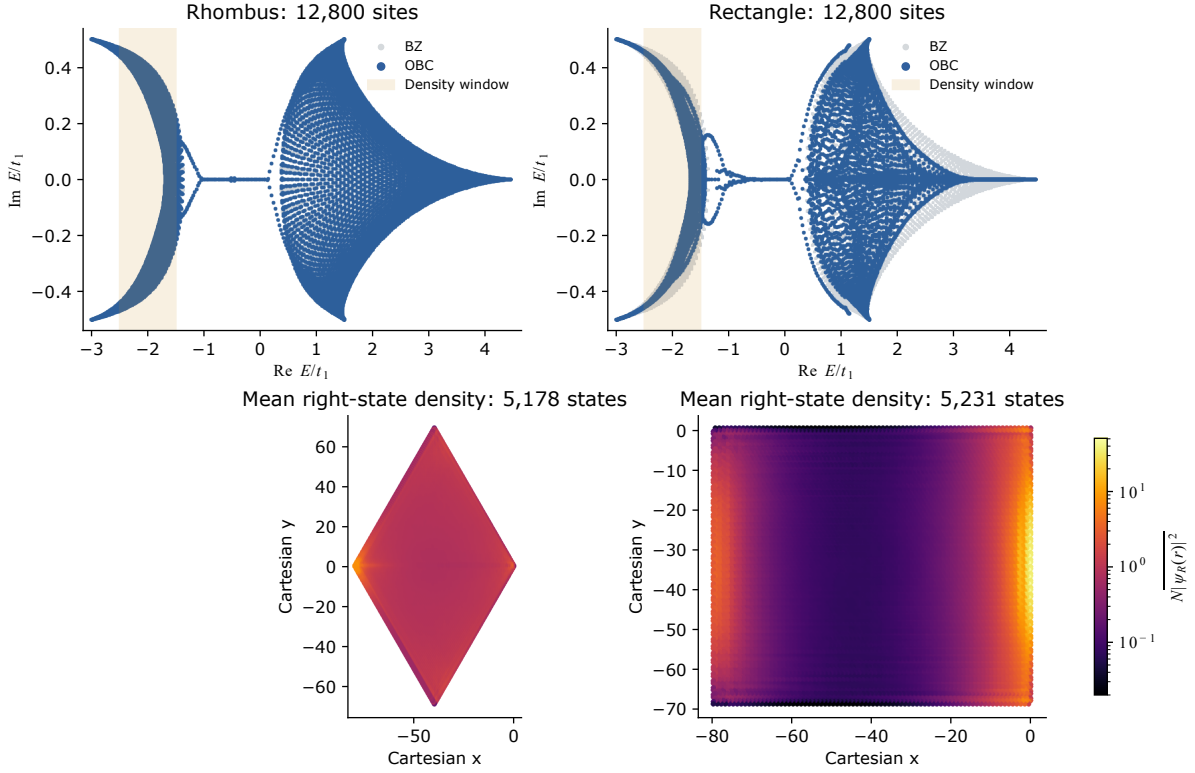


Figure 4: Existing OBC data for an 80×80 primitive-cell rhombus and an 80×40 four-site-cell rectangle, both containing 12,800 sites. Top: finite OBC spectra and the BZ reference. Gap states are included in these spectra. Bottom: mean normalized right-state intensity in the shaded window $-2.5 \leq \text{Re } E \leq -1.5$, containing 5,178 and 5,231 states, respectively. Both maps share a logarithmic color scale; a uniform density would have value one. The rectangle exhibits pronounced accumulation near its armchair sides, whereas the rhombus retains substantial bulk weight. Finite-size modulations and corner enhancement remain visible in the rhombus.

The finite spectra need not coincide point by point with the infinite-strip spectra: the order of limits differs, and finite OBC spectra also contain boundary states. Their role here is to check the geometric construction and the spatial localization predicted by the directional comparison.

State density and data provenance

Every right eigenvector is normalized separately before averaging. For the selected set $\mathcal{W}$, the plotted quantity is

$$\rho(r) = \frac{N}{|\mathcal{W}|} \sum_{n \in \mathcal{W}} \frac{|\psi_{R,n}(r)|^2}{\sum_{r'} |\psi_{R,n}(r')|^2}, \quad \frac{1}{N} \sum_r \rho(r) = 1.$$

The energy window lies inside the lower bulk band and avoids the central topological gap. It is a reproducible energy selection, not a rigorous topological classification of each finite-system state. Averaging many such states reduces the risk of mistaking a single topological corner mode for extensive bulk skin localization. The compact archives also retain the all-state mean, inverse participation ratios, and one deterministically selected state (nearest to $-2 + 0.2i$ within the same window).

Sources of the figures

- **SGBZ:** the four supplied files `data/Haldane-gain-loss-{a1,a2,x,y}-SGBZ.pkl`. The script checks the energy-grid ordering and model parameters when reading them. The two-site scans have 26 polynomial terms; the four-site scans have 64.
- **OBC rhombus:** `paper-Haldane-gain-loss-OBC-80-80.pkl` from `myoffice:~/654/research-data/2D_skin_effect/phcpy-free-algorithm/data`.
- **OBC rectangle:** `paper-Haldane-gain-loss-OBC-square-80-40.pkl` from the same directory. The legacy name “square” refers to the Cartesian cell construction; the actual finite sample need not be a geometric square.
- **Compact extracts:** `obc-rhombus.npz` and `obc-rectangle.npz` beside the figures. Each retains all eigenvalues and site coordinates together with normalized density summaries. The original eigenvector arrays are not duplicated. Source paths, file sizes, modification times, and extraction times are stored in each archive’s `metadata` field.

The legacy OBC tuples contain no model-parameter metadata. Their parameter assignment follows the companion Haldane script supplied with those calculations. The original caches are preserved; the figures represent reused research data, not fresh large-scale calculations in this example.

Validation performed for this example

The `check` command compares the BerryPy characteristic polynomial with $\det(EI - h)$ at eight deterministic complex arguments per direction. Maximum relative errors are below 3.3×10^{-15} . Independently, direct finite-lattice assembly is compared with BerryPy’s two-stage supercell construction followed by `get_bulk_Hamiltonian_complex((None, None))`: all four matrices agree exactly, with coordinate differences below 4.5×10^{-16} .

The small demonstration samples six energies per direction, including spectral interior, exterior, and a real-energy continuum case, and diagonalizes two 128-site OBC samples. It tests the complete pipeline; those small lattices are not used to claim thermodynamic convergence. The detailed validation record, including unexpected results, accompanies this case in `log/2026-09-25-geometry-dependent-skin-effect.md`.

Running and extending the example

Run the commands below from the repository root. Multiline examples use POSIX shell continuation (`\`); in PowerShell, enter the command on one line or use its backtick continuation. The script resolves default directories relative to its own location. Install this checkout with `python -m pip install -e .`, and use an environment containing BerryPy, SymPy, NumPy, SciPy, and Matplotlib. These application dependencies are separate from the lightweight `pygbz2d` package dependencies.

■ Small end-to-end calculation

```
python application/geometry-dependent-skin-effect.py demo --workers 2
python application/geometry-dependent-skin-effect.py check
```

The demo writes to `application/data/geometry-dependent-skin-effect/demo`. Its grid is $\text{Re } E \in \{-2, 1.5, 5\}$ and $\text{Im } E \in \{0, 0.3\}$. Intermediate results are saved after each energy. A repeat invocation reuses completed SGBZ results; OBC diagonalization and figures are regenerated.

■ Larger SGBZ scans on a server

```
python application/geometry-dependent-skin-effect.py sweep \
--directions a1 a2 x y --n-real 201 --n-imag 201 --workers 12
```

The default sweep directory is `application/data/geometry-dependent-skin-effect/sweep`. The default energy bounds match the old scans. Reuse exactly the same command to resume missing entries; add `--retry-failed` to recompute failed entries. A checkpoint with incompatible direction, grid, polynomial, or parameters is rejected. Choose another `--output-dir` when changing the run. The default `sweep` grid, if sizes are omitted, is only 7×5 .

■ Finite OBC calculations

```
python application/geometry-dependent-skin-effect.py obc \
--geometry rhombus --nx 80 --ny 80 \
--output application/data/geometry-dependent-skin-effect/rhombus.npz
python application/geometry-dependent-skin-effect.py obc \
--geometry rectangle --nx 80 --ny 40 \
--output application/data/geometry-dependent-skin-effect/rectangle.npz
```

Add `--save-vectors` to retain the full eigensystem in a neighboring `.pkl` file. Without it, only the compact summary is saved. Dense diagonalization scales cubically in the number of sites, and one $12,800 \times 12,800$ complex matrix alone occupies about 2.44 GiB; the eigensolver needs additional workspace. Start with `--nx 8 --ny 8` when checking a new environment.

| Reusing results and rebuilding this note

■ Extracting old OBC data where it is stored

The `summarize-obc` command reads the tuple `(eigenvalues, right_eigenvectors, coordinates)` produced by the old script. It processes eigenvector columns in batches to avoid allocating another full square probability matrix. This extraction path needs NumPy but does not import BerryPy or `pygbz2d`.

```
python application/geometry-dependent-skin-effect.py summarize-obc \
/path/to/paper-Haldane-gain-loss-OBC-80-80.pkl \
--output obc-rhombus.npz
```

Run extraction on the data server and copy the small `.npz` result for plotting. For streaming over SSH, `--output -` writes a binary NPZ archive to standard output. Use a binary-safe client when saving that stream. Pickle inputs must come from trusted calculations.

■ Recreating the figures from the supplied caches

```
python application/geometry-dependent-skin-effect.py plot --obc \
application/Figures/geometry-dependent-skin-effect/obc-rhombus.npz \
application/Figures/geometry-dependent-skin-effect/obc-rectangle.npz
```

This reads the SGBZ files from the repository's `data` directory and writes SVG, PNG, and a JSON numerical summary to `application/Figures/geometry-dependent-skin-effect`. Add `--sgbz-dir application/data/geometry-dependent-skin-effect/sweep` to visualize a newly computed scan. `--directions y` restricts the plot to one direction; `--nk` controls the BZ reference sampling. Plotting performs no SGBZ root solving or OBC diagonalization.

```
typst compile application/geometry-dependent-skin-effect.typ
```

The compiled note uses the checked-in figures. For a convergence study, increase the energy-grid resolution and the OBC sizes separately, retain the same density window across sizes, and inspect both logarithmic radii and real-space distributions. Small isolated radius deviations should remain visible in the numerical record until their cause is understood.

| References

- [1] Y. Li, C. Liang, C. Wang, C. Lu, and Y.-C. Liu, Gain-Loss-Induced Hybrid Skin-Topological Effect, *Phys. Rev. Lett.* **128**, 223903 (2022).
- [2] C. Wang, J. Pi, Q. Liu, Y. Li, and Y.-C. Liu, General Theory for Geometry-Dependent Non-Hermitian Bands, *Arxiv* (2025).

Online versions: *Phys. Rev. Lett.* 128, 223903 (2022) [arXiv:2506.22743v3](https://arxiv.org/abs/2506.22743v3).